

Microsoft 365 Copilot — why the GitHub Copilot interception approach doesn't transfer

Written 2026-09-21, as a follow-up to [copilot-desktop-apps-coverage-gap.html](#) 's disambiguation of the word "Copilot." That doc ruled Microsoft 365 Copilot **out of scope permanently** for a simple reason: it's a different company's product, talking to a different backend, with no shared infrastructure to intercept via anything built for GitHub Copilot. This doc answers a narrower, separate question raised afterward: **forget GitHub entirely — could the same *kind of approach* (a local listener a client is redirected to, relaying to the real backend) work against Microsoft 365 Copilot on its own terms?**

Short answer: no, for two independent reasons, checked against Microsoft's own current admin documentation, not guessed. Either one alone would be enough to close this; both being true makes it a firmer close than the VS Code Copilot Chat case, which at least had one real, if gated, lever to chase.

What Microsoft 365 Copilot actually is

Microsoft's own public description: an AI assistant integrated into Word, Excel, PowerPoint, Outlook, Teams, Loop, and Whiteboard. A user's prompt is grounded in their actual organizational data via **Microsoft Graph**, then sent to a large language model on **Azure OpenAI Service**, with the response grounded back into the document/app context. Confirmed distinct from GitHub Copilot on Microsoft's own page: *"a completely separate product from GitHub Copilot."* Structurally the same shape as GitHub Copilot — client app → orchestration/auth layer → LLM backend → response — but a completely different, Microsoft-owned backend with zero relationship to GitHub's CAPI.

Reason 1: there is no redirect lever — the endpoint is fixed, not configurable

GitHub Copilot's VS Code integration had [github-enterprise.uri](#): a real [settings.json](#) key that (for an enterprise-flagged account) determines where the client sends its discovery call, and whose response the client trusts unconditionally for where to send everything after. That's the lever the whole interception mechanism was built on.

Microsoft 365 Copilot has no equivalent. Per Microsoft's own admin documentation ([App and network requirements for Microsoft Copilot admins](#)):

- Copilot's enterprise experiences connect to a fixed set of domains: `copilot.cloud.microsoft`, more broadly `*.cloud.microsoft`, and `*.office.com`. These are the same for every tenant — not resolved from a per-tenant or per-device setting the way `github-enterprise.uri` is.
- Microsoft's explicit guidance to admins is to allow the **entire** `*.cloud.microsoft` domain, stating directly: "*Microsoft doesn't support allowing partial or only selected Microsoft 365 application URLs within the `*.cloud.microsoft` domain.*" That's a statement about network policy, but it doubles as confirmation there's no narrower, single addressable Copilot-specific endpoint an admin (or a config write) could isolate and redirect — the whole wildcard domain is the unit Microsoft expects you to treat atomically.
- No config file, registry key, environment variable, or admin policy surfaced in this documentation that repoints where any of this traffic goes. Compare this to VS Code, where `github-enterprise.uri` is a real, documented, user-or-workspace-scoped `settings.json` key that does exactly that (for the one account type that reads it).

Consequence: there's no equivalent of "write one config key, watch the client connect to us instead." Nothing here determines its own destination from anything write-accessible. This alone is sufficient to close the question — everything below is a second, independent problem that would still remain even if this one were somehow solved.

Reason 2: the traffic is WebSockets, not request/response HTTP — and Microsoft documents interception breaking it

Every interception mechanism this codebase has actually built — `kinds::copilot_discovery`'s direct call-and-rewrite, the `forward()` /Optimus tunnel every `BaseUrl` agent uses — is shaped around discrete HTTP request/response pairs: a request comes in, gets relayed or answered, a response goes back. Microsoft 365 Copilot's traffic is not that shape.

Per the same admin doc's **WebSockets (WSS) protocol requirements** section:

Verify that your network supports full WSS connectivity from user devices running Microsoft 365 applications to the following domains: Microsoft Copilot enterprise experiences: `*.office.com`, `*.cloud.microsoft` and `copilot.cloud.microsoft`.

And, listed as a typical cause of "*Copilot application failures*":

Network devices attempting to perform Transport Layer Security (TLS) inspection of connections

A related doc on Copilot's newer "Cowork" agentic feature ([Cowork network endpoints](#)) is more direct about the failure mode:

SSL/TLS inspection isn't supported and might interrupt long-lived SSE connections, which can result in message delivery failures, retries, or frozen tasks.

What this does and doesn't prove. This is Microsoft describing what happens with *generic corporate TLS-inspecting proxy appliances* — hardware typically tuned for short-lived HTTP request/response, which often mishandle long-lived streaming connections (aggressive idle timeouts, buffering that breaks keep-alives). It is not a claim that no relay could ever handle this traffic cleanly — a purpose-built proxy that correctly holds a WebSocket open, passes frames through without buffering, and imposes no idle timeout is a solved problem in general (this is how API gateways and service meshes routinely handle WebSocket traffic). No evidence of application-level certificate pinning (an absolute technical block, distinct from a reliability complaint) was found in this pass — the language throughout is about connection fragility under naive inspection, not a stated refusal to accept a foreign certificate.

What remains genuinely unverified: whether this codebase's own relay infrastructure (`forward()` /Optimus, or a hypothetical new listener kind) is even shaped to proxy a long-lived WebSocket without the failure modes Microsoft describes. Everything built so far is request/response-shaped. That would be new, real engineering work, not a reuse of anything that exists today — and it would only be worth attempting after Reason 1 is somehow solved, which it currently isn't.

Net assessment

	GitHub Copilot (VS Code)	Microsoft 365 Copilot
Redirect lever	<code>github-enterprise.uri</code> — real, but gated by the <code>enterprise: true</code> account flag (§3.1 of the VS Code write-up)	None found — fixed, wildcarded, tenant-wide domain
Traffic shape	HTTP request/response — matches existing relay infrastructure	WebSockets (WSS) — a different shape than anything built so far
Documented interception risk	None found for the mechanism itself	Microsoft explicitly documents TLS inspection causing failures on this traffic
Verdict	Mechanism proven correct with real traffic; account-gate is the one open, hard-to-close gap	Closed — no known foothold, and a second, harder engineering problem sits behind where the first one would have been

Recommendation: don't pursue this further under the current approach. The VS Code Copilot Chat case at least has one concrete, real mechanism with a single named gap (the `enterprise:true` account flag) worth watching for a future closure. Microsoft 365 Copilot has no comparable foothold at all — closing Reason 1 would require Microsoft shipping some new, currently-nonexistent redirect capability, and closing Reason 2 would still require solving a WebSocket-relay engineering problem this codebase has never attempted. Revisit only if Microsoft's own network requirements documentation changes to describe a real, addressable, redirectable endpoint for Copilot traffic specifically.

Sources

- [App and network requirements for Microsoft Copilot admins](#) — Microsoft Learn, updated 2026-09-08
- [Microsoft Copilot Cowork network endpoints \(Preview\)](#) — Microsoft Support
- [Microsoft 365 URLs and IP address ranges](#) — Microsoft Learn